



中山大學
SUN YAT-SEN UNIVERSITY

异常处理

中山大学计算机类专业教程



主讲人：郑贵锋



联系方式：

zhenggf@mail.sysu.edu.cn

目录

CONTENTS

01

问题引出

c 风格异常表示, c 标准库变量与函数

02

异常处理概述

03

异常定义与抛出

04

异常的捕获及处理

05

异常编程与实践



C语言错误处理回顾 (Review)

程序错误

- 编译 或 静态分析 (搜索: c++ 静态分析) 错误
 - 语法错误 (无法编译)
 - 逻辑错误, 习惯不良 (New-Delete不配对, Define-Use关系如使用前未初始化, ...)
- 运行错误
 - 运行**异常**
 - 可以检测: 内存分配失败, 文件读写错误, 除零, 指针操作越界...
 - 难以预测: 段错误, 栈溢出 ...
 - 逻辑**错误**
 - 程序正常运行, 但未得到期望的结果



C语言错误处理回顾 (Review)

C语言错误表示与处理

- 一般地，如果函数返回 int 或 pointer
 - 0 表示成功，非零表示错误编码
 - 空指针表示失败，非空表示正常
- 使用相关全局变量与函数，例如：对于标准库：
 - `errno` `<error.h>`
 - `char *strerror(int errnum)` `<string.h>`
 - `void perror(char *string)` `<stdio.h>`



C语言错误处理回顾 (Review)

C语言处理案例代码：

c_error.c

```
int main(){
    FILE * pf;
    int errnum;
    pf = fopen("unexist.txt", "rb");
    if (pf == NULL) {
        errnum = errno; //防止被其他错误覆盖
        fprintf(stderr, "error num: %d\n", errno);
        perror("open(unexist.txt)");
        fprintf(stderr, "open file error: %s\n", strerror( errnum ));
    }
    else {
        // 继续做文件操作
        fclose (pf);
    }
    return 0;
}
```



C语言错误处理回顾 (Review)

处理异常，提升程序的健壮性(Robustness)

```
double Cal_scores(int score)
{
    if (score < 0 || score > 100)
    {
        cout << " XXXX ";    //处理异常
        return -1;
    }
    else
    {
        :                //正常代码
    }
}
```

健壮性是指一个计算机系统在执行过程中处理错误，以及算法在遭遇输入、运算等异常时保持正常运行的能力。



C语言错误处理回顾 (Review)

- 优势

- 简单，高效

- 缺点

- 应检必检。必须检测各种错误可能，对编程能力与经验要求极高！
- 检测到错误，必须及时地合理地处理
- 程序中散布大量错误处理代码，正常代码与错误处理代码混在一起，难以阅读与维护

Why use exceptions?

The basic answer is: **Using exceptions for error handling makes your code simpler, cleaner, and less likely to miss errors.** But what's wrong with "good old errno and if-statements"? The basic answer is: Using those, your error handling and your normal code are closely intertwined. That way, your code gets messy and it becomes hard to ensure that you have dealt with all errors (think "spaghetti code" or a "rat's nest of tests").

目录

CONTENTS

01

问题引出

02

异常处理概述

异常概念，C++异常处理基本思想

03

异常定义与抛出

04

异常的捕获及处理

05

异常编程与实践



异常处理概述

- 程序终止

- 执行正常结束而终止。
- 程序执行中发生错误或特殊事件而终止（异常终止）。
 - 可预测的错误
 - 用户自己定义的错误
 - 难以预测的错误

- 比拟场景，例如：

- 我开着车，一头猪冲到路中间，刹车 - 异常（可挽救）
- 我开着车，发动机歇火，停车 - 故障（系统崩溃）



异常处理概述

异常处理（exception handling）机制的基本思想

- 采用结构化方法对程序的运行时错误进行显式管理：
 - 处理的是可预料的错误或特殊事件。
 - 将程序中的正常处理代码与异常处理代码显式区别开来，提高程序的可读性。



C++语言中的异常处理

基本思想：

- 结构化定义异常
 - 将异常种类定义为树状结构
- 结构化处理异常（**异常检测与异常处理分离**）
 - 异常检测部分检测到异常的存在时，抛出一个异常对象给异常处理代码。
 - 在程序或函数特定位置，集中捕获异常对象，再处理异常。
 - 通过该异常对象，独立开发的异常检测部分和异常处理部分能够就程序执行期间所出现的异常情况进行通信



C++语言中的异常处理

异常用于给出错误信号，并不必立即处理。例如：

```
class VectorInSpecialMemory {
    int sz;
    int* elem;
public:
    VectorInSpecialMemory(int s)
        : sz(s)
        , elem(AllocateInSpecialMemory(s))    //对象分配空间可能失败
    {
        if (elem == nullptr)                //检测错误
            throw std::bad_alloc();          //抛出异常，如何处理这时并不一定知道
    }
    ...
};
```



C++语言中的异常处理

抛出异常与异常处理分离的实现：

抛出异常的程序段

...

`throw` 表达式;

...

捕获并处理异常的程序段

`try`

复合语句（保护段）

`catch`（异常类型声明）

复合语句

`catch`（异常类型声明）

复合语句

...



C++语言中的异常处理

Try-Catch 执行机制

- 若程序有异常，则通过 throw 关键字创建一个异常对象，并抛出。
- 将可能抛出异常的程序段嵌在 try 块保护段之中。控制通过正常的顺序执行到达 try 块，然后执行 try 子块内的保护段。
- 如果在保护段执行期间没有引发异常，那么跟在try子块后的catch子句就不执行。程序继续执行紧跟在try块中最后一个catch子句后面的语句。
- 如果有异常
 - catch子句按其在try块后出现的顺序被检查。类型匹配的catch子句将捕获并处理异常（或继续抛出异常）。
 - 如果找不到匹配的处理代码，则自动调用标准库函数terminate，其默认功能是调用abort() 终止程序。

目录

CONTENTS

01

问题引出

02

异常处理概述

03

异常定义与抛出

Throw关键字，异常类型，常见标准库异常，异常树

04

异常的捕获及处理

05

异常编程与实践



C++异常定义与 throw 关键字

throw 关键字

throw 表达式; (1)

throw; (2) 用于当前异常再抛出

- 表达式可以是任意类型的对象
- **throw 表达式；** 操作的基本行为：
 1. 复制被抛出的对象为临时对象，作为异常对象
 2. 如果在当前函数 **try** 块中，则中止 try 执行，执行 **catch** 语句捕获
 3. 如有异常对象未在当前函数被捕获处理，则中止函数执行，异常**传播**给调用者处理



C++异常定义与 throw 关键字

案例，

c_error_cpp.c

```
#include <stdio>
#include <cerrno>

int main () {
    FILE * pf;
    try {
        pf = fopen ("unexist.txt", "rb");
        if (pf == NULL) throw errno;
        // write something;
        fclose (pf);
    }
    catch (int errnum) {
        if (errnum!=2) fclose (pf);
        else perror("unexist.txt");
    }
    return 0;
}
```

int 作为异常:

1. 易于用 c 库错误代码，易于代码向 C++迁移
2. 实现检测与处理代码分离

注：针对异常资源的管理，RAII (*Resource Acquisition Is Initialization*) 是现代C++设计的基本技术。后面有介绍



C++异常定义与 throw 关键字

标准库异常基类 exception 的定义

头文件: <exception>

```
class exception {  
public:  
    exception () noexcept;  
    exception (const exception&) noexcept;  
    exception& operator= (const exception&) noexcept;  
    virtual ~exception();  
    virtual const char* what() const noexcept;  
}
```

作为各种异常的基类: (1) 虚函数将提供什么能力? (2) 虚析构呢? (3) 猜猜关键字 noexcept 的含义。



C++异常定义与 throw 关键字

阅读参考 `exception` `<exception>`

- `exception` 有哪些已知的子类（从多个头文件中收集）
- 哪些子类有已知的子类
- 请画出它们的继承关系

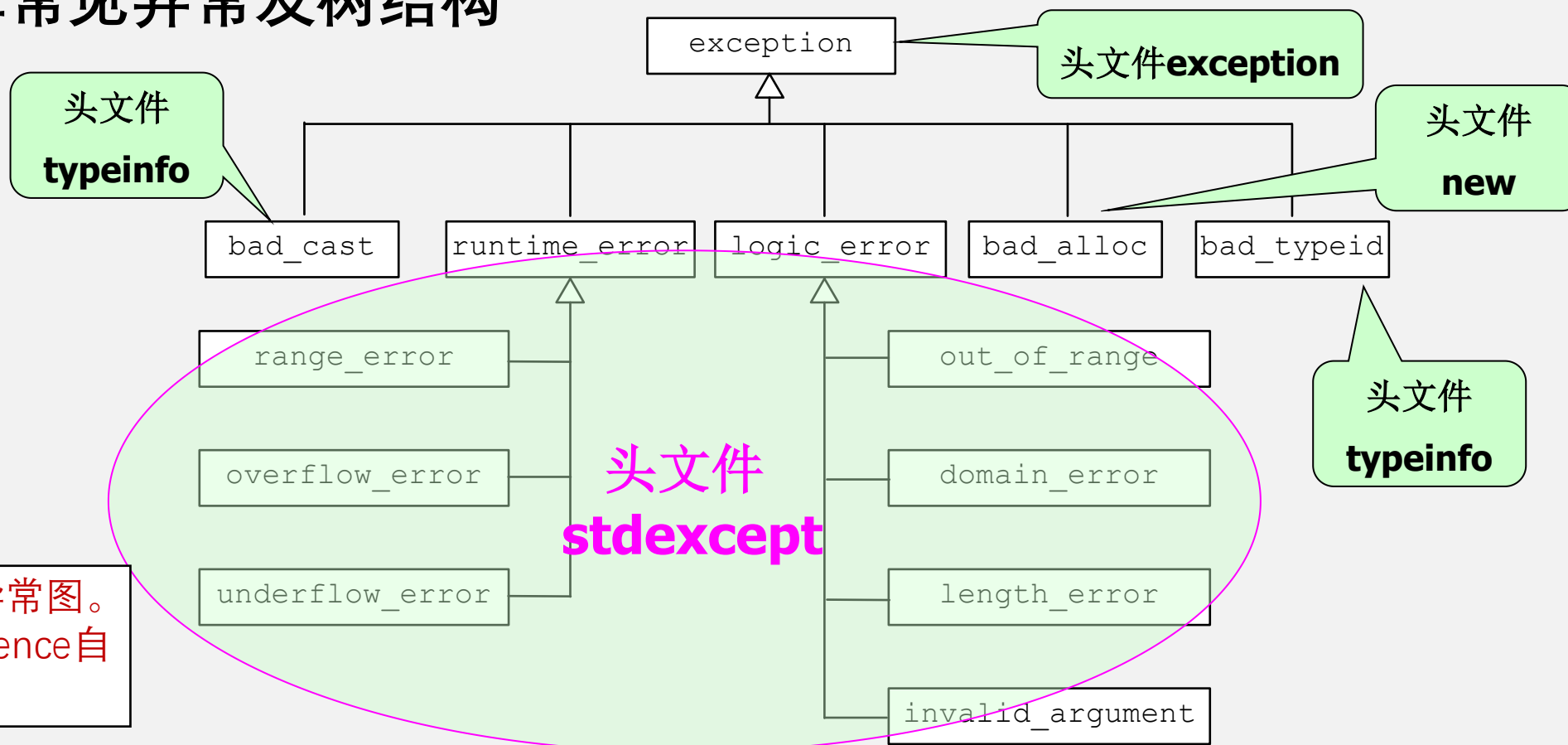
例如：

- `std::bad_alloc` `<new>`
- `std::runtime_error` `<stdexcept>`
- `std::logic_error` `<stdexcept>`



C++异常定义与 throw 关键字

标准库常见异常及树结构



这就旧版C++异常图。
请根据cppreference自己完善



C++异常定义与 throw 关键字

使用标准异常

exception-bad-alloc.cpp

```
// bad_alloc example
#include <iostream>          // std::cout
#include <new>                // std::bad_alloc

int main () {
    try
    {
        int* myarray= new int[10000];
    }
    catch (std::bad_alloc& ba)
    {
        std::cerr << "bad_alloc caught: " << ba.what() << '\n';
    }
    return 0;
}
```

C++异常定义与 throw 关键字

logic_error 定义（是逻辑类异常的公共基类）

```
class logic_error : public exception {  
public:  
    explicit logic_error (const string& what_arg);  
    explicit logic_error (const char* what_arg);  
};
```

- 关键字 explicit（为什么要用？）
- 定义了构造函数，让 what() 显示需要的内容。
- out_of_range 定义





C++异常定义与 throw 关键字

使用Out-Of-Range异常

exception-out-of-range.cpp

```
// out_of_range example
#include <iostream>           // std::cerr
#include <stdexcept>          // std::out_of_range
#include <vector>              // std::vector

int main (void) {
    std::vector<int> myvector(10);
    try {
        myvector.at(20)=100;    // vector::at throws an out-of-range
    }
    catch (const std::out_of_range& oor) {
        std::cerr << "Out of Range error: " << oor.what() << '\n';
    }
    return 0;
}
```

目录

CONTENTS

01

问题引出

02

异常处理概述

03

异常定义与抛出

04

异常的捕获及处理

异常捕获语法，捕获规则与多态，异常传播，异常说明，重抛异常

05

异常编程与实践



C++异常捕获与处理

异常处理语法结构

```
try {  
    program-statements //程序的正常处理逻辑  
    throw exception-object //程序的正常处理逻辑  
    program-statements  
    ...  
}  
catch (exception-declaration) {  
    handler-statements //异常处理代码  
}  
catch (exception-declaration) {  
    handler-statements //异常处理代码  
}  
...  
catch (...) {  
    handler-statements //异常处理代码  
}
```

try 块中一旦遇到异常，便终止当前块执行。

try 块后至少有一个异常申明。例如：
exception& e

异常申明。异常对象
建议用引用，如
string&等

使用...处理前面未截获的其他异常，必须放在最后



C++异常捕获与处理

案例：参数异常与捕获

exception-argument.cpp

```
double mySqrt(double dnum)
{
    if (dnum < 0)
        throw std::invalid_argument("argument dnum must be great than zero");
    return std::sqrt(dnum); //对合法参数进行处理
}
```

```
int main() {
    try {
        mySqrt(-1);
    }
    catch (std::exception& e) {
        std::cout << e.what() << '\n';
    }
    return 0;
}
```

if语句对参数异常进行检测，并抛出异常

- 使用引用输出：
argument dnum must be great than zero
- 不使用引用输出：
std::exception
为什么？



C++异常捕获与处理

异常捕获的匹配规则

- try 期间发生异常;
- 按 catch 顺序与异常定义匹配
 - 异常对象类型是定义类型或是定义类型的子类
 - 匹配成功
 - 异常接收变量是值, 复制异常到接收变量
 - 异常接收变量是引用或指针, 创建引用和指针
 - 清除当前异常, 执行该异常处理块
- 直到匹配成功或结束

要点:

- 子类必须优先基类catch, (…) 必须是最后catch
- 用基类引用类型捕捉异常, 且虚函数才能产生多态



C++异常捕获与处理

案例：异常捕获顺序与多态练习

exception-derived-without-virtual.cpp

```
class EBase {  
    //EX1: 添加 virtual 比较输出不同  
public: void what() { cout << "base" << endl; };  
};  
class EDriven:public EBase {  
public: void what() { cout << "EDrived" << endl; };  
};  
  
int main() {  
    try {  
        throw EDriven();  
    }  
    //注意：捕获的次序，子类必须在前面。EX2: 消除编译警告  
    catch (EBase& e) { e.what();}  
    catch (EDrived& e) { e.what();}  
    return 0;  
}
```

编译警告：基类
earlier...



C++异常捕获与处理

异常传播 (exception propagation)

- 从异常抛出到控制转移给合适的异常处理语句的过程就叫做异常传播。
- 异常传播过程（一句话：当前函数未处理异常，则交给调用函数）：
 - 由里层向外层的执行每一个包围抛出点（异常被抛出的最初位置）的try语句。
 - 如果当前的成员函数调用中没能定位异常处理，则调用终止。并且在该成员函数调用点将该异常抛给调用者，重复执行上一步。
 - 如果该异常终止了当前线程或进程的所有成员函数调用，则说明该线程或进程中不存在对异常的处理，它将自行终止。



C++异常捕获与处理

案例：异常传播（重要）



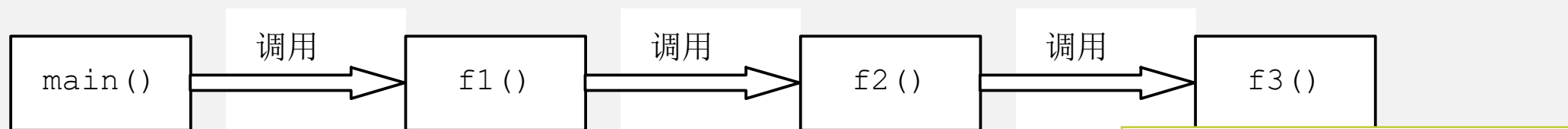
```
void f3(int x) {  
    switch (x) {  
        case 1: throw 3.4;           // 抛出double型异常  
        case 2: throw 2.5f;         // 抛出float型异常  
        case 3: throw 1;            // 抛出int型异常  
        case 5: throw exception(); // 抛出标准异常  
    }  
    cout << "End of f3" << endl;  
}
```

exception-propagation.cpp



C++异常捕获与处理

案例：异常传播（续）



```
void f2(int x) {  
    try {  
        f3(x);  
    }  
    catch (int) {          //int型异常的处理代码  
        cout << "An int exception occurred!--from f2" << endl;  
    }  
    catch (float) {        //float型异常的处理代码  
        cout << "A float exception occurred!--from f2" << endl;  
    }  
    cout << "End of f2" << endl;  
}
```

exception-propagation.cpp



C++异常捕获与处理

案例：异常传播（续）

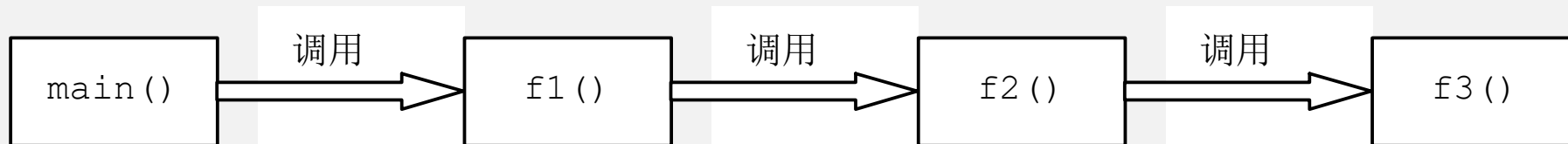
exception-propagation.cpp

```
void f1(int x) {  
    try {  
        f2(x);  
    }  
    catch (int) {                // int型异常的处理代码  
        cout << "An int exception occurred!--from f1" << endl;  
    }  
    catch (float) {             // float型异常的处理代码  
        cout << "A float exception occurred!--from f1" << endl;  
    }  
    catch (double) {            // double型异常的处理代码  
        cout << "A double exception occurred!--from f1" << endl;  
    }  
    cout << "End of f1" << endl;  
}
```



C++异常捕获与处理

案例：异常传播（续）



```
int main()
{
    for (int i = 1; i < 6; i++) {
        cout << "----- input i = " << i << endl;
        f1(i);
    }

    cout << "End of main" << endl;
    return 0;
}
```

exception-propagation.cpp

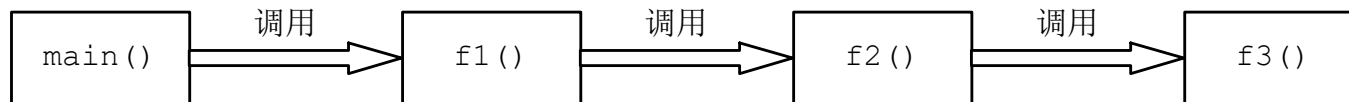


C++异常捕获与处理

案例：异常传播（结果）

exception-propagation.cpp

```
----- input i = 1
A double exception occurred!--from f1
End of f1
----- input i = 2
A float exception occurred!--from f2
End of f2
End of f1
----- input i = 3
An int exception occurred!--from f2
End of f2
End of f1
----- input i = 4
End of f3
End of f2
End of f1
----- input i = 5
terminate called after throwing an instance of 'std::exception'
what():  std::exception
```





C++异常捕获与处理

异常说明 (except specification) / (限定函数异常)

```
void f3(int x) throw(double, float) {  
    switch (x) {  
        case 1: throw 3.4;           // 抛出double型异常  
        case 2: throw 2.5f;         // 抛出float型异常  
        case 3: throw 1;            // 抛出int型异常  
        case 5: throw exception(); // 抛出标准异常  
    }  
    cout << "End of f3" << endl;  
}
```

exception-propagation.cpp

添加函数异常限定
C++11警告, **17**取消

限定以外的异常, 直接
调用**std::abort()**



C++异常捕获与处理

noexcept 关键字

```
void f2(int x) noexcept {  
    try {  
        f3(x);  
    }  
    catch (int) {          //int型异常的处理代码  
        cout << "An int exception occurred!--from f2" << endl;  
    }  
    catch (float) {        //float型异常的处理代码  
        cout << "A float exception occurred!--from f2" << endl;  
    }  
    cout << "End of f2" << endl;  
}
```

exception-propagation.cpp

C++11开始，考虑到优化与编程实际需要，仅需说明是否允许抛出异常

有异常未处理，则直接调用**std::abort()**



C++异常捕获与处理

异常重新抛出

- 抛出该catch子句原来所捕获的异常，由函数调用链中更上层的函数来处理该异常。

```
catch (MyException &e) {  
    e.setMessage("new message");  
    throw;  
}
```

修改被捕获的异常对象；导致传递到上层函数的MyException类型异常对象的数据成员message中所包含的字符串将变成“new message”。

```
catch (MyException e) {  
    e.setMessage("new message");  
    throw;  
}
```

修改被捕获异常对象的局部副本；传递到上层函数的MyException类型异常对象并不发生改变。

目录

CONTENTS

01

问题引出

02

异常处理概述

03

异常定义与抛出

04

异常的捕获及处理

05

异常编程与实践

自定义异常，对象构造异常，RAII技术简介



C++异常编程与实践

用户自定义异常

- 强力建议从<stdexcept>包继承异常，常用基类有：
- logic_error
- runtime_error
- 需要错误 **code** 属性， system_error< system_error >

优势

- 代码更简洁，易于阅读
- 更利于分类处理异常
- 减少自定义虚函数（多数情况标准库都做好了）



C++异常编程与实践

用户自定义异常（案例）

exception-divided-by-zero.cpp

```
class Divided_By_Zero:public runtime_error{
public:
    explicit Divided_By_Zero(const string& s = "Divide By Zero"):runtime_error(s){};
    explicit Divided_By_Zero(const char* s):runtime_error(string(s)){};
};

int main() {
    double a = 10, b = 0, res;
    char Operator = '/';
    try {
        if (b == 0) throw Divided_By_Zero();
        res = a / b;
        cout << a << " / " << b << " = " << res;
    }
    catch(const exception& e) {
        cout << e.what() << endl;
    }
    return 0;
}
```



C++异常编程与实践

思考题

- 用户希望给出异常发生的时刻。例如：
- Divided_By_Zero::what() 应返回
- “2021/06/01-09:28:02 Divide By Zero”
- 请问应如何修改程序满足需求？



C++异常编程与实践

对象构造异常及案例

exception-constructor.cpp

```
class Dummy {  
public:  
    ~Dummy() {  
        std::cout << "Dummy destructed\n";  
    }  
};
```

正常构造的**自动（局部）对象**，
会在退出作用域时**自动析构**

```
class Triangle {  
public:  
    Triangle(int a=3,int b=4,int c=5) {  
        if (a < 0) { throw a;}  
    }  
    ~Triangle() {  
        std::cout << "Triangle destructed\n";  
    }  
private:  
    int a,b,c;  
};
```

构造函数异常的对象，**不会分**
配空间，也不会析构。



C++异常编程与实践

对象构造异常及案例（续）

exception-constructor.cpp

```
int main() {  
    try {  
        Dummy dum;  
        //if new or alloc something?  
        Triangle tri(-1);  
        std::cout << "Triangle created\n";  
    }  
    catch (...) {  
        std::cout << "Exception caught\n";  
    }  
}
```

```
> ./a  
Dummy destructed  
Exception caught
```



C++异常编程与实践

RAII (Resource Acquisition Is Initialization) 技术 (选讲)

中文翻译为资源获取即初始化。即使用自动对象来管理资源的技术。资源主要是指操作系统中有限的东西如内存、文件等等，利用自动对象的生命周期特征，自动管理资源。如**C++文件**，**智能指针**等

整个RAII技术总结四个部分：

- a) 设计一个类封装资源
- b) 在构造函数中初始化，申请资源
- c) 在析构函数中执行销毁操作
- d) 使用时声明一个该类的局部对象



C++异常编程与实践

RAII技术与传统方法对比

```
void fct(string s)
{
    File_handle f(s,"r");    // File_handle's constructor opens the file called "s"
    // use f
} // here File_handle's destructor closes the file
```

```
void old_fct(const char* s)
{
    FILE* f = fopen(s,"r"); // open the file named "s"
    // use f
    fclose(f); // close the file
}
```

C++异常编程与实践

RAII技术应用 – 智能指针

- 用特殊的对象 Wrap 指向资源的指针，当这些对象在析构时，自动释放指向的资源。

Managed pointers

auto_ptr	Automatic Pointer (deprecated) (class template)
auto_ptr_ref	Reference to automatic pointer (class template)
shared_ptr C++11	Shared pointer (class template)
weak_ptr C++11	Weak shared pointer (class template)
unique_ptr C++11	Unique pointer (class template)
default_delete C++11	Default deleter (class template)



C++异常编程与实践

RAII技术应用 – 智能指针示例

```
// auto_ptr example
#include <iostream>
#include <memory>

int main () {
    std::auto_ptr<int> p1 (new int);
    *p1.get()=10;

    std::auto_ptr<int> p2 (p1);

    std::cout << "p2 points to " << *p2 << '\n';
    // (p1 is now null-pointer auto_ptr)

    return 0;
}
```



C++异常编程与实践

RAII技术应用 – 智能指针自学

- **Managed Pointer**

- `unique_ptr`
 - 只允许基础指针的一个所有者。
- `shared_ptr`
 - 采用引用计数的智能指针。
 - 一个原始指针分配给多个所有者，直至所有 `shared_ptr` 所有者超出了范围或放弃所有权，才会删除原始指针。
- `weak_ptr`
 - 结合 `shared_ptr` 使用的特例智能指针。

智能指针（现代 C++）

<https://docs.microsoft.com/zh-cn/cpp/cpp/smart-pointers-modern-cpp>



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



中山大学MOOC课程组